



PIMPRI CHINCHWAD EDUCATION TRUST'S.
PIMPRI CHINCHWAD COLLEGE OF ENGINEERING
(An Autonomous Institute)

B. TECH

Year: 2026 – 27

Semester: VII

Name of the Student: Swayam Mandhani

PRN: 123B1B184

Department: Computer Engineering

Division: C, **Batch** – C2

Course: DevOps

Course Code: BCE27PE01

Assignment No – 3

Title:

Discover Infrastructure as Code using Terraform and write a Terraform script to create an AWS EC2 virtual machine.

Aim:

To study and implement Infrastructure as Code (IaC) using Terraform by defining and provisioning an AWS EC2 instance and its associated security group through Terraform configuration files.

Theory:

Infrastructure as Code (IaC) is an approach for managing and provisioning computing infrastructure through machine-readable configuration files rather than manually configuring infrastructure.

Terraform is a declarative Infrastructure as Code tool used to define, provision, and manage infrastructure resources. In this assignment, Terraform was used with the AWS provider to provision an Ubuntu EC2 instance and a security group in the Mumbai region.

The practical workflow was: Terraform configuration → terraform init → terraform validate → terraform plan → terraform apply → infrastructure verification → terraform destroy.

Code / Implementation:

Project Structure

Assignment-3/

```
|— terraform/  
|  |— provider.tf  
|  |— variables.tf  
|  |— main.tf  
|  |— outputs.tf  
|  |— .gitignore  
|  └─ terraform.tfvars  
|— screenshots/  
└─ README.md
```

provider.tf

```
terraform {  
  required_version = ">= 1.5.0"  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = "~> 6.0"  
    }  
  }  
}  
  
provider "aws" {  
  region = var.aws_region  
}
```

variables.tf

```
variable "aws_region" {  
  description = "AWS region where the EC2 instance will be created"  
  type        = string  
  default     = "ap-south-1"
```

```

}

variable "instance_type" {
    description = "EC2 instance type"
    type        = string
    default     = "t3.micro"
}

variable "ami_id" {
    description = "Ubuntu AMI ID for the EC2 instance"
    type        = string
}

variable "key_name" {
    description = "Name of the AWS EC2 key pair"
    type        = string
}

variable "instance_name" {
    description = "Name tag for the EC2 instance"
    type        = string
    default     = "DevOps-Terraform-EC2"
}

resource "aws_security_group" "devops_sg" {
    name        = "devops-terraform-sg"
    description = "Security group for DevOps Terraform EC2 instance"
    ingress {
        description = "SSH"
        from_port   = 22
        to_port     = 22
        protocol    = "tcp"
        cidr_blocks = ["0.0.0.0/0"]
    }

    ingress {
        description = "HTTP"
        from_port   = 80
        to_port     = 80
    }
}

```

```
protocol    = "tcp"
cidr_blocks = ["0.0.0.0/0"]
}
```

```
egress {
  description = "Allow all outbound traffic"
  from_port   = 0
  to_port     = 0
  protocol    = "-1"
  cidr_blocks = ["0.0.0.0/0"]
}
```

```
tags = {
  Name       = "DevOps-Terraform-SG"
  Assignment = "DevOps-Assignment-03"
  ManagedBy  = "Terraform"
}
}
```

```
resource "aws_instance" "devops_ec2" {
  ami           = var.ami_id
  instance_type = var.instance_type
  key_name      = var.key_name
  vpc_security_group_ids = [aws_security_group.devops_sg.id]
```

```
tags = {
  Name       = var.instance_name
  Assignment = "DevOps-Assignment-03"
  ManagedBy  = "Terraform"
}
}
```

outputs.tf

```
output "instance_id" {  
    description = "ID of the EC2 instance"  
    value      = aws_instance.devops_ec2.id  
}  
  
output "instance_public_ip" {  
    description = "Public IP address of the EC2 instance"  
    value      = aws_instance.devops_ec2.public_ip  
}  
  
output "instance_public_dns" {  
    description = "Public DNS name of the EC2 instance"  
    value      = aws_instance.devops_ec2.public_dns  
}  
  
output "security_group_id" {  
    description = "ID of the security group"  
    value      = aws_security_group.devops_sg.id  
}
```

AWS Configuration Used

Parameter	Actual Value
AWS Region	ap-south-1
AMI	Ubuntu 24.04 LTS
AMI ID	ami-007b1f3fdea0383d9
Instance Type	t3.micro
Key Pair	devops-assignment-2-key
Security Group	devops-terraform-sg
SSH Port	22
HTTP Port	80

Implementation Steps

Step 1 — AWS authentication

aws configure

aws sts get-caller-identity

Step 2 — Initialize Terraform

```
terraform init
```

Step 3 — Format and validate

```
terraform fmt
```

```
terraform validate
```

Step 4 — Generate plan

```
terraform plan
```

Step 5 — Provision infrastructure

```
terraform apply
```

Step 6 — Verify EC2

Verified the Terraform-created EC2 instance in the AWS Management Console.

Step 7 — Verify outputs and state

```
terraform output
```

```
terraform state list
```

Step 8 — Verify NGINX

```
sudo apt update
```

```
sudo apt install nginx -y
```

```
sudo systemctl enable --now nginx
```

Step 9 — Cleanup

```
terraform destroy
```

Output

The following screenshots were captured during the actual implementation. Insert the corresponding screenshot at each indicated location.

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Admin> terraform version
Terraform v1.15.8
on windows_amd64

Your version of Terraform is out of date! The latest version
is 1.16.4. You can update by downloading from https://developer.hashicorp.com/terraform/install
PS C:\Users\Admin> |

```

Figure 1: Terraform version verification

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS D:\Projects\DevOps Github\Assignment-3> cd terraform
PS D:\Projects\DevOps Github\Assignment-3\terraform> dir

Directory: D:\Projects\DevOps Github\Assignment-3\terraform

Mode                LastWriteTime         Length Name
----                -
d-----          9/25/2026   9:39 PM             .terraform
-a-----          9/25/2026   9:34 PM           347 .gitignore
-a-----          9/25/2026   9:39 PM        1481 .terraform.lock.hcl
-a-----          9/25/2026   9:39 PM        1204 main.tf
-a-----          9/25/2026   9:33 PM         527 outputs.tf
-a-----          9/25/2026   9:32 PM         207 provider.tf
-a-----          9/25/2026   9:45 PM        8976 terraform.tfstate
-a-----          9/25/2026   9:45 PM       3108 terraform.tfstate.backup
-a-----          9/25/2026   9:44 PM         174 terraform.tfvars
-a-----          9/25/2026   9:33 PM         621 variables.tf

PS D:\Projects\DevOps Github\Assignment-3\terraform> |

```

Figure 2: Terraform project files

```
Problems Output Debug Console Terminal Ports powershell - terraform + v @ | | ... |
PS D:\Projects\DevOps Github\Assignment-3\terraform> terraform plan
    + "0.0.0.0/0",
    ]
    + description      = "SSH"
    + from_port        = 22
    + ipv6_cidr_blocks = []
    + prefix_list_ids  = []
    + protocol         = "tcp"
    + security_groups  = []
    + self             = false
    + to_port          = 22
  },
]
+ name                = "devops-terraform-sg"
+ name_prefix         = (known after apply)
+ owner_id            = (known after apply)
+ region              = "ap-south-1"
+ revoke_rules_on_delete = false
+ tags                = {
    + "Assignment" = "DevOps-Assignment-03"
    + "ManagedBy" = "Terraform"
    + "Name"       = "DevOps-Terraform-SG"
  }
+ tags_all            = {
    + "Assignment" = "DevOps-Assignment-03"
    + "ManagedBy" = "Terraform"
    + "Name"       = "DevOps-Terraform-SG"
  }
+ vpc_id              = (known after apply)
}

Plan: 2 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ instance_id          = (known after apply)
+ instance_public_dns  = (known after apply)
+ instance_public_ip   = (known after apply)
+ security_group_id    = (known after apply)

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to
take exactly these actions if you run "terraform apply" now.
PS D:\Projects\DevOps Github\Assignment-3\terraform> |
```

Figure 3: Terraform execution plan

```
PS D:\Projects\DevOps Github\Assignment-3\terraform> terraform apply
+ network_interface (known after apply)
+ primary_network_interface (known after apply)
+ private_dns_name_options (known after apply)
+ root_block_device (known after apply)
+ secondary_network_interface (known after apply)
}

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ instance_id = (known after apply)
+ instance_public_dns = (known after apply)
+ instance_public_ip = (known after apply)

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

aws_instance.devops_ec2: Creating...
aws_instance.devops_ec2: Still creating... [00m10s elapsed]
aws_instance.devops_ec2: Creation complete after 12s [id=i-0213e369ad43d98d3]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

instance_id = "i-0213e369ad43d98d3"
instance_public_dns = "ec2-65-0-73-47.ap-south-1.compute.amazonaws.com"
instance_public_ip = "65.0.73.47"
security_group_id = "sg-04ae315878da8108b"
PS D:\Projects\DevOps Github\Assignment-3\terraform>
```

Figure 4: Successful Terraform apply

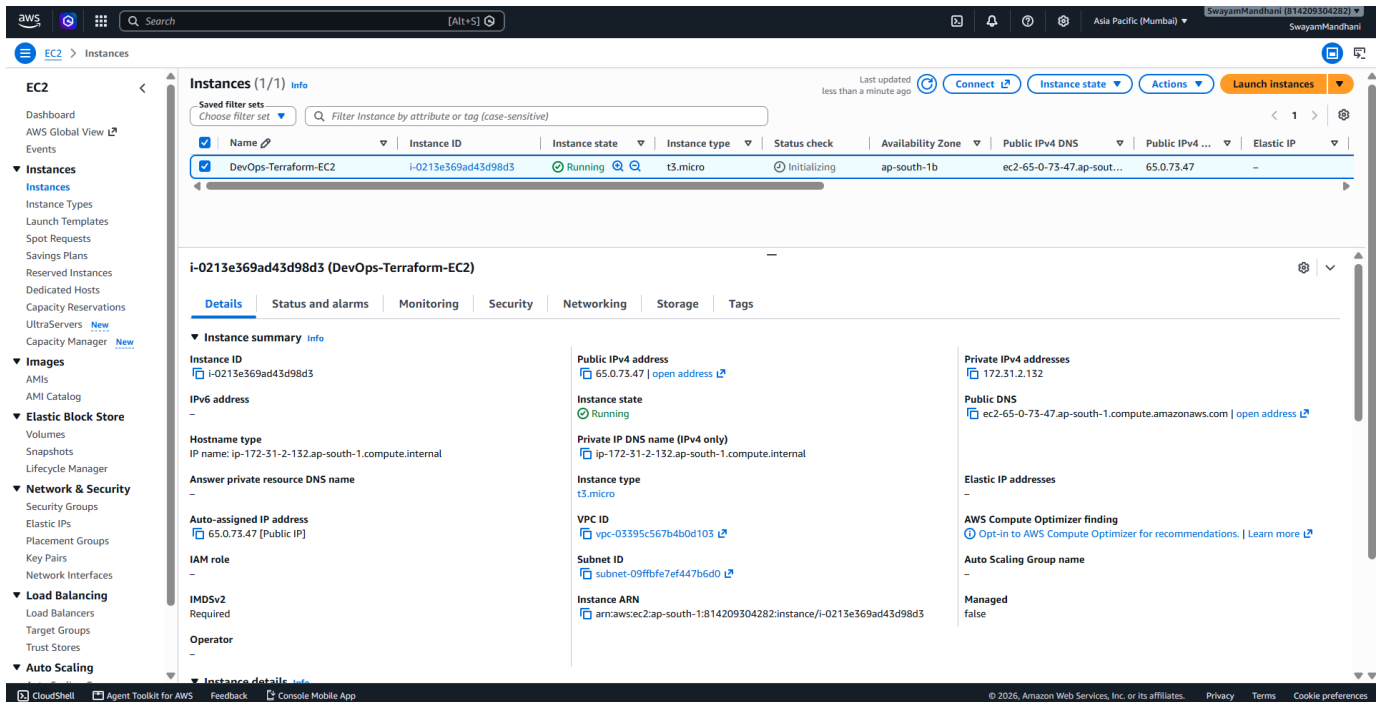


Figure 5: Terraform-provisioned EC2 instance running in AWS

```
PS D:\Projects\DevOps Github\Assignment-3\terraform> terraform output
instance_id = "i-0213e369ad43d98d3"
instance_public_dns = "ec2-65-0-73-47.ap-south-1.compute.amazonaws.com"
instance_public_ip = "65.0.73.47"
security_group_id = "sg-04ae315878da8108b"
PS D:\Projects\DevOps Github\Assignment-3\terraform> terraform state list
aws_instance.devops_ec2
aws_security_group.devops_sg
PS D:\Projects\DevOps Github\Assignment-3\terraform> |
```

Figure 6: Terraform outputs and state

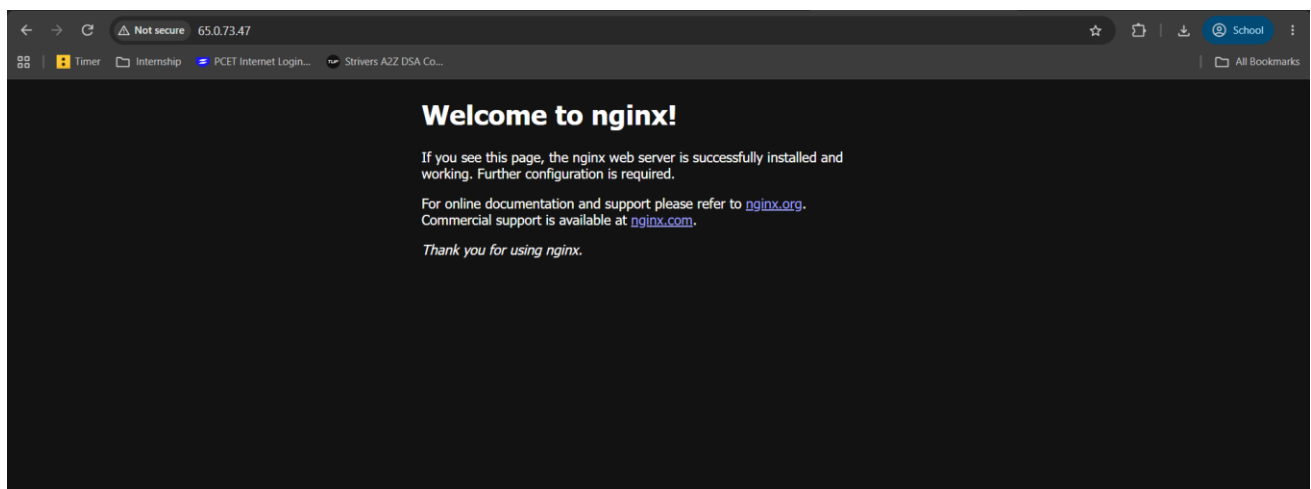


Figure 7: NGINX web page running on the provisioned EC2 instance

```

        - self                = false
        - to_port              = 22
    },
] -> null
- name                        = "devops-terraform-sg" -> null
- owner_id                   = "814209304282" -> null
- region                     = "ap-south-1" -> null
- revoke_rules_on_delete     = false -> null
- tags                       = {
    - "Assignment" = "DevOps-Assignment-03"
    - "ManagedBy" = "Terraform"
    - "Name"       = "DevOps-Terraform-SG"
} -> null
- tags_all                  = {
    - "Assignment" = "DevOps-Assignment-03"
    - "ManagedBy" = "Terraform"
    - "Name"       = "DevOps-Terraform-SG"
} -> null
- vpc_id                    = "vpc-03395c567b4b0d103" -> null
# (1 unchanged attribute hidden)
}

```

Plan: 0 to add, 0 to change, 2 to destroy.

Changes to Outputs:

```

- instance_id                = "i-0213e369ad43d98d3" -> null
- instance_public_dns        = "ec2-65-0-73-47.ap-south-1.compute.amazonaws.com" -> null
- instance_public_ip         = "65.0.73.47" -> null
- security_group_id          = "sg-04ae315878da8108b" -> null

```

Do you really want to destroy all resources?

Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

```

aws_instance.devops_ec2: Destroying... [id=i-0213e369ad43d98d3]
aws_instance.devops_ec2: Still destroying... [id=i-0213e369ad43d98d3, 00m10s elapsed]
aws_instance.devops_ec2: Still destroying... [id=i-0213e369ad43d98d3, 00m20s elapsed]
aws_instance.devops_ec2: Destruction complete after 30s
aws_security_group.devops_sg: Destroying... [id=sg-04ae315878da8108b]
aws_security_group.devops_sg: Destruction complete after 1s

```

Destroy complete! Resources: 2 destroyed.

PS D:\Projects\DevOps Github\Assignment-3\terraform> |

Figure 8: Successful Terraform infrastructure cleanup

Result:

The AWS infrastructure was successfully provisioned using Terraform. The EC2 instance and security group were defined as code, the EC2 instance was verified, NGINX was accessed through HTTP, Terraform outputs and state were inspected, and the Terraform-managed infrastructure was successfully destroyed after completion.

Conclusion:

In this assignment, I learned the concept of Infrastructure as Code and implemented it using Terraform. I configured the AWS provider, defined variables and outputs, created an AWS security group and EC2 instance, validated and planned the infrastructure, and provisioned the resources using Terraform. I also verified the deployed EC2 instance and NGINX web server, examined Terraform state, and performed resource cleanup using terraform destroy. This practical provided hands-on understanding of automated cloud infrastructure provisioning and lifecycle management.

References:

1. Terraform Documentation — <https://developer.hashicorp.com/terraform/docs>
2. AWS EC2 Documentation — <https://docs.aws.amazon.com/ec2/>
3. AWS CLI Documentation — <https://docs.aws.amazon.com/cli/>
4. AWS IAM Documentation — <https://docs.aws.amazon.com/iam/>
5. AWS Security Groups Documentation —
<https://docs.aws.amazon.com/vpc/latest/userguide/vpc-security-groups.html>

Submission Links:

GitHub Repository Link: <https://github.com/SwayamMandhani06/DevOps/tree/main/Assignment-3>

Hosted / Output Link: NA – AWS Console based practical